ICT2232 – SOFTWARE ENGINEERING

MINI PROJECT GUIDELINES

1. Project Overview

This mini project requires each group to identify a real-world problem, analyse the needs of relevant stakeholders, define and prioritise requirements, design a solution, implement and test a working software system, deploy it, respond to a change request, and address security concerns. The project is designed to give you practical experience of the Software Development Life Cycle (SDLC).

The emphasis is not only on producing a working application. Your group must demonstrate how decisions and artefacts develop from one SDLC stage to the next.

2. Learning Objectives

- Apply software engineering principles to a realistic software problem.
- Identify stakeholders, business needs, assumptions, constraints and project scope.
- Elicit, analyse, specify, prioritise and validate functional and non-functional requirements.
- Apply an appropriate software process model and Agile practices where appropriate.
- Transform requirements into an appropriate software design and architecture.
- Implement a working software solution using suitable technologies.
- Plan and perform appropriate levels of software testing.
- Deploy the developed system in an accessible environment.
- Analyse and respond to a software change request as a maintenance activity.
- Identify security risks and implement appropriate security controls.
- Work effectively as a software development team and demonstrate individual contribution.

3. Project Allocation

The following project domain is assigned to each group. Groups should develop the specified system for the stated target users. The listed features are starting requirements; groups are expected to refine and prioritise them during requirements engineering.

Group	Project	Purpose
Group 1	Campus Lost & Found Management System	A platform for students and staff to report, search, verify and recover lost property.
Group 2	Student Club & Society Management System	A system for managing university clubs, memberships, activities and participation.
Group 3	Home Services Booking Platform	A platform that connects customers with service providers such as electricians, plumbers and cleaners.
Group 4	Pet Care & Veterinary Appointment System	A system for managing pet profiles, veterinary appointments and care records.
Group 5	School Transport Management System	A system for managing school transport routes, vehicles, drivers and student transport arrangements.
Group 6	Restaurant Table & Queue Management System	A system that helps a restaurant manage reservations, walk-in customers and table availability.
Group 7	Blood Donation Camp Management System	A system for organising blood donation campaigns, donor registrations, appointments and stock information.
Group 8	Apartment Visitor & Facility Management System	A system for managing residents, visitors, shared facilities and service requests in an apartment

		complex.
Group 9	Small Business Invoice & Expense Management System	A lightweight system for small businesses to manage customers, invoices, expenses and financial summaries.
Group 10	University Internship Management System	A system for coordinating students, internship opportunities, employers, supervisors and internship progress.
Group 11	Community Food Donation Coordination System	A platform for coordinating surplus food donations between donors, collection volunteers and community organisations.
Group 12	Equipment Rental Management System	A system for an organisation that rents equipment such as cameras, projectors, tools or event equipment.
Group 13	Childcare Centre Management System	A system for managing children, guardians, attendance, activities, staff schedules and communication.
Group 14	Freelancer Project Management Platform	A platform for clients and freelancers to manage projects, tasks, milestones, deliverables and payments.
Group 15	Public Service Complaint & Resolution System	A system for citizens to submit service complaints, track progress and receive resolution updates.

Note: The feature lists above are not final requirements. Your group must analyse the problem domain and produce its own validated, prioritised requirements.

4. General Project Requirements

- Each group must develop a working software application for its assigned domain.
- The system must contain multiple related modules and a persistent data store where appropriate.
- Groups may choose the programming language, framework, database and development tools.
- The selected technologies must be justified in the project documentation.
- Each group member must make a meaningful contribution across the SDLC and must be able to explain the work they completed.
- The project must contain both functional requirements and non-functional requirements.
- The system must be tested using documented test cases and test results.
- The final application must be deployed in an environment that can be demonstrated to the lecturer.
- Groups must maintain version control throughout development.
- AI-assisted development tools may be used where permitted, but students remain responsible for understanding, verifying, testing and documenting generated output.

5. SDLC Project Roadmap

Your project must progress through the following stages. The outputs from one stage should inform the next stage.

SDLC Stage	Required Evidence
Stage 1 – Problem & Project Definition	Problem statement, background, stakeholders, objectives, scope, assumptions and constraints.
Stage 2 – Process Planning	Selected software process model, justification, project plan and development approach.
Stage 3 – Agile Planning	Product backlog, user stories, priorities, acceptance criteria, sprint plan and task allocation.
Stage 4 – Requirements Engineering	Functional requirements, non-functional requirements, use cases, prioritisation and traceability.
Stage 5 – Design	System architecture, UML/design

	diagrams, database design and user-interface prototype.
Stage 6 – Implementation	Working software, database, validation, error handling and integration of major modules.
Stage 7 – Testing	Test plan, test cases, execution results, defect identification and correction.
Stage 8 – Deployment	Deployed application, deployment architecture, installation/configuration instructions and demonstration.
Stage 9 – Maintenance	Change request, impact analysis, modification plan, updated artefacts and implemented change where feasible.
Stage 10 – Security	Security threats, risk assessment, security requirements and implemented security controls.

6. Requirements Engineering Deliverables

Your requirements document must contain:

- Stakeholder list and stakeholder needs.
- System scope: what is included and excluded.
- At least 12 well-formed functional requirements.
- At least 6 non-functional requirements covering relevant categories such as performance, usability, reliability, security and maintainability.
- At least 10 user stories using an appropriate format.
- Acceptance criteria for the major user stories.
- Use case diagram and descriptions for important use cases.
- Prioritisation of requirements using a suitable technique such as MoSCoW.
- A requirements traceability matrix connecting requirements to design, implementation and tests.

7. Design Deliverables

- High-level system architecture.
- Use case diagram.
- Class/domain model or equivalent design representation.
- At least two sequence diagrams for important system interactions.
- At least one activity/process diagram for a significant workflow.
- Database/ER diagram and major database tables where a database is used.
- Basic user-interface prototype or wireframes for major screens.
- Design decisions and brief justification of important architectural or technology choices.

8. Implementation Requirements

- Implement the prioritised core requirements.
- Demonstrate complete end-to-end workflows for important use cases.
- Use appropriate coding standards and meaningful naming.
- Use version control and maintain a useful commit history.
- Implement input validation and appropriate error handling.
- Use appropriate authentication and authorisation where the domain requires different user roles.
- Do not claim functionality as complete if it is only a mock-up or static screen.

9. Testing Requirements

- Prepare a test plan based on the requirements.
- Create test cases with test ID, related requirement, preconditions/input, expected result, actual result and status.
- Include positive and negative test cases.
- Demonstrate unit, integration, system and acceptance testing at an appropriate level for the project.
- Record defects found and explain how important defects were corrected.
- Ensure final test results can be traced back to requirements.

10. Deployment Requirements

- Deploy the application to a suitable hosting environment or other demonstrable deployment environment.
- Provide the deployment URL or deployment package where applicable.
- Document system requirements and configuration.
- Provide installation/deployment steps.
- Explain the deployed architecture and database configuration.
- Demonstrate the deployed system during the final evaluation.

11. Maintenance Activity

After the initial version is completed, the lecturer will provide or approve a change request. Your group must treat it as a real maintenance task.

- Describe the requested change.
- Identify affected requirements.
- Perform an impact analysis covering design, database, code, testing and documentation where relevant.
- Explain the maintenance approach and estimate the work involved.
- Update the affected SDLC artefacts.
- Implement and test the change where feasible.
- Explain whether the change represents corrective, adaptive, perfective or preventive maintenance.

12. Security Requirements

- Identify at least five security threats relevant to your application.
- Assess the likelihood and impact of important threats.
- Define security-related requirements.
- Implement at least three appropriate security controls.
- Demonstrate protection against relevant issues such as unauthorised access, weak authentication, insecure input handling or inappropriate access to data.
- Explain security decisions made during design and implementation.

13. Teamwork and Individual Contribution

- Each member must contribute to requirements, design, implementation and/or testing activities; contribution should not be limited to documentation or presentation preparation.
- Maintain a contribution log identifying major tasks completed by each member.
- Use version control to provide evidence of collaborative development.
- Each member must individually demonstrate and explain selected project work during the final presentation/viva.
- The lecturer may ask any member to explain requirements, design decisions, code, test cases or security measures.

14. Suggested Project Timeline

Milestone	Focus	Expected Evidence
Milestone 1	Project definition & process model	Problem statement, stakeholders, scope, objectives, selected process model
Milestone 2	Agile planning	Product backlog, user stories, priorities, acceptance criteria, sprint plan
Milestone 3	Requirements baseline	FRs, NFRs, use cases, prioritisation, traceability
Milestone 4	Design baseline	Architecture, UML/design diagrams, database design, UI prototype
Milestone 5	Implementation & testing	Working core system, test cases and results
Milestone 6	Deployment	Deployed system and deployment documentation
Milestone 7	Maintenance & security	Change request analysis, updated system/artefacts, security assessment
Final	Presentation & demonstration	Complete SDLC evidence and working software

15. Final Submission List

- Project report documenting the SDLC activities and decisions.
- Requirements specification and traceability matrix.
- Agile/product backlog and sprint evidence.
- Design document and diagrams.
- Source code repository or submitted source code.

- Working deployed application.
- Testing documentation and test evidence.
- Maintenance change request and impact analysis.
- Security assessment and evidence of implemented controls.
- User guide and technical/deployment guide.
- Individual contribution record.
- Final presentation/demo.

16. Recommended Final Project Report Structure

1. 1. Cover Page
2. 2. Introduction and Problem Domain
3. 3. Problem Statement
4. 4. Stakeholders and Objectives
5. 5. Scope, Assumptions and Constraints
6. 6. Software Process Model and Justification
7. 7. Agile Planning
8. 8. Requirements Engineering
9. 9. System Design
10. 10. Implementation
11. 11. Testing and Quality Assurance
12. 12. Deployment
13. 13. Maintenance and Change Request
14. 14. Security
15. 15. Project Management and Individual Contributions
16. 16. Conclusion
17. 17. References
18. 18. Appendices

17. Assessment Rubric – 20 Marks

Area	Marks	What is assessed
Problem definition & process planning	2	Clarity of problem, objectives, stakeholders, scope, assumptions and process-model justification.
Agile planning & project management	1	Backlog quality, user stories, prioritisation, sprint planning and evidence of team

		coordination.
Requirements engineering	3	Quality, completeness, correctness, prioritisation and traceability of requirements.
System design	2	Appropriateness and consistency of architecture, UML/design models, database and UI design.
Implementation & functionality	4	Working functionality, integration, code quality, validation, error handling and completeness of core workflows.
Testing & quality assurance	2	Test planning, coverage, execution, defect handling and traceability.
Deployment	2	Successful deployment, configuration/documentation and demonstration of the deployed system.
Maintenance/change request	2	Quality of impact analysis and handling of the requested change.
Security	1	Threat analysis, security requirements, controls and evidence of implementation.
Final presentation, demonstration & individual contribution	1	Clarity of presentation, quality of demonstration, teamwork and ability of each member to explain their contribution.

18. Final Demonstration Expectations

Your final presentation should demonstrate the evolution of the project, not only the finished application.

- Briefly explain the problem and target stakeholders.
- Show selected requirements and user stories.
- Explain the selected development process and Agile approach.
- Show the key design artefacts and explain important design decisions.
- Demonstrate the working application using realistic fictional data.
- Demonstrate important test cases and outcomes.
- Show the deployed system.
- Explain the maintenance change request and how your system responded.
- Demonstrate or explain the implemented security controls.

- Each member must explain their own contribution and answer questions.